Generative Ontology Induction: Domain-Agnostic Schema Discovery from Document Corpora Using Large Language Models

Sergei Sergienko[1] (ORCID: 0009-0009-5180-7115)*

Pivots Global
ssergienko@ontology.live

Abstract.. Ontology engineering remains a critical bottleneck in knowledge-intensive AI systems. Existing automated approaches either depend on predefined schemas, operate within narrow domains, or produce unstructured outputs unsuitable for downstream pipelines.
We introduce **Generative Ontology Induction (GOI)**, a domain-agnostic framework that induces a *generative blueprint*-entities, dimensions, properties, relationships, and constraints-from a corpus of examples and exports it as a typed graph (six node types, seven edge types) in YAML/JSON. We introduce the **Node Coverage Score**, a novel evaluation metric that measures the percentage of ontology dimension nodes appearing in generated outputs.
A controlled generative validation on two contrasting ontologies-a familiar Software Services Invoice schema and a custom Job Description Ontology-shows that GOI-prompted generation by construction covers 100% of applicable ontology nodes; a generic three-field template recovers 95.7% on the invoice schema but collapses to 65.4% on the Job Description Ontology. The structural coverage holds regardless of how familiar the document type is to the model.

Keywords: Ontology learning · Schema induction · Large language models · Knowledge graphs · Domain-agnostic methods · Generative models · Cross-functional teams

1 Introduction

Modern AI systems increasingly rely on structured knowledge to ground reasoning, guide generation, and ensure consistency across complex workflows. Ontologies-formal representations of entities, relationships, and constraints within a domain-serve as the backbone for knowledge graphs, semantic search, retrieval-augmented generation (RAG), and multi-agent orchestration. Yet ontology engineering remains a labor-intensive, expert-driven process that creates a fundamental bottleneck: every new domain, document type, or application requires manual schema design, iterative refinement, and domain-specific rules.

This **structure gap** manifests across the AI landscape. Enterprise knowledge management systems struggle to maintain consistent schemas as document types evolve. Research teams building domain-specific knowledge graphs invest months in ontology design before extraction can begin. Agent frameworks that promise autonomous operation still require hand-crafted schemas to structure their memory and reasoning. The promise of general-purpose AI collides with the reality that structured knowledge remains stubbornly domain-specific and manually curated.

A particularly underexplored dimension of this gap occurs within mixed-expertise teams: researchers, backend engineers, and frontend developers operate with fundamentally different mental models, and lack a shared, human-readable structural representation of the product's knowledge layer.

Recent advances in large language models (LLMs) have begun to address the structure gap. Zero-shot and few-shot ontology construction pipelines demonstrate that LLMs can extract entities, induce taxonomies, and generate schema elements without extensive training data [1,2,3]. Corpus-based methods can discover hierarchical event ontologies from open-domain text collections [4]. Interactive systems enable human-guided schema discovery from research corpora [5]. Yet these approaches share common limitations: they often require predefined ontology seeds, operate within constrained domains, produce untyped or inconsistently structured outputs, lack robust cross-document canonicalization, and provide limited pathways for non-expert users to visualize, validate, and export discovered schemas.

This paper introduces **Generative Ontology Induction (GOI)**, a framework designed to address these gaps through four core contributions:

First, we formalize the concept of *generative ontology*-the structural schema that defines a document class and enables generation of new instances-and distinguish it from descriptive entity extraction. GOI

* An earlier version of this work was previously deposited on Zenodo (DOI: 10.5281/zenodo.19772179).

reverse-engineers this generative blueprint by analyzing multiple examples of the same document type, identifying recurring dimensions, properties, relationships, and constraints that govern the class.

Second, we present a complete technical architecture comprising a universal typed graph representation (six node types, seven edge types), a multi-document prompt that elicits generative schemas rather than per-document entities, an automatic type-based layout algorithm, multi-format import (OWL, RDF, YAML, JSON, Markdown), and a dual-view YAML/JSON export whose `promptReady` markdown rendering realizes the "ontology drives AI pipelines" thesis by allowing direct injection into RAG and agent prompts without graph traversal.

Third, we introduce the **Node Coverage Score**, a novel evaluation metric that measures the percentage of dimension nodes in an induced ontology that appear in outputs generated from that ontology-a direct structural-completeness signal that token-level metrics miss.

Fourth, we identify and address a practical **cross-functional team communication gap**: mixed-expertise AI product teams lack a shared, readable structural artifact, and GOI's visual, interactive, exportable schema serves as precisely this common artifact, positioning GOI as both a technical contribution and an **organizational primitive**.

A controlled generative validation on two contrasting ontologies demonstrates GOI's structurally-mandated coverage; we also discuss limitations including LLM consistency, token-window constraints, the lack of formal OWL axioms, and hallucination risks.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 details the GOI framework. Section 4 introduces the Node Coverage Score. Section 5 presents experimental results. Section 6 discusses implications and limitations. Section 7 concludes.

2 Related Work

2.1 Zero-Shot and Few-Shot Schema Induction

LLM-driven methods now produce cross-domain schemas and corpus-level ontologies without heavy supervision. Corpus-based open induction uses distant supervision and corpus-wide signals to induce hierarchical event ontologies without direct supervision [4]. Zero-shot generation of sources synthesizes or retrieves pseudo-documents to expose latent schema structure and then induces schema elements in a zero-shot manner [2]. Schema-from-text pipelines that do not assume a predefined schema perform extraction then define and canonicalize schema elements post-hoc, enabling domain-agnostic operation [6]. Prompt-only competitive methods apply LLM prompting-including retrieval-augmented prompts and sampling strategies-to construct terms, types, and taxonomies across domains without model fine-tuning [1]. OntoGenix [8] shows that LLMs can be directly leveraged for ontology engineering from tabular datasets, generating reusable conceptual models. Recent benchmarking efforts such as Text2KGBench [23] have begun to standardize evaluation of ontology-driven knowledge graph generation from text, surfacing the need for structural-not just token-level-evaluation criteria.

2.2 Knowledge Graph Construction from Document Collections

iText2KG [11] processes document collections incrementally, using a graph integrator to merge entity and relation extractions across documents into a unified knowledge graph. The system explicitly acknowledges that unresolved semantically duplicate entities remain a challenge. AutoClusRE [7] builds corpus-level type tables guided by dynamically updated entity and relation type tables, using semantic clustering to reduce duplication across texts. Iterative zero-shot prompting pipelines [3] offer scalable corpus-level extraction without training examples. Graph-based event schema induction [15] builds event graphs and clusters them to induce recurring event templates across documents. SAC-KG [24] treats LLMs as automatic constructors of domain-specific knowledge graphs, demonstrating that prompt-driven pipelines can scale beyond hand-crafted extractors. At the application layer, Microsoft's GraphRAG [22] highlights how downstream retrieval-augmented systems benefit from explicit graph-structured knowledge over raw text indices, motivating the need for high-quality schemas of the kind GOI induces.

2.3 Ontology Learning with LLMs

Lo et al. [9] present an end-to-end ontology learning pipeline with LLMs and introduce structural and semantic graph distance measures as evaluation metrics, noting that standard token-level metrics miss structural correctness and coherence. Beliaeva and Rahmatullaev [1] survey heterogeneous LLM methods

for ontology learning, identifying canonicalization and evaluation as open problems. Sadruddin et al. [5] introduce LLMs4SchemaDiscovery, a human-in-the-loop system that converts research questions plus a corpus into a structured schema and grounded database. OntoKGen [12] provides an adaptive iterative Chain-of-Thought interface for ontology extraction, integrating generated knowledge graphs into Neo4j for flexible querying and visual exploration. NeOn-GPT [21] combines a methodology framework with LLM prompts to produce Turtle ontologies, finding that LLMs reduce effort but lack procedural reasoning capabilities. OntoGPT's SPIRES system [17] extracts semantic structures according to LinkML schemas without training data, demonstrating schema-guided extraction patterns. ReCG [16] applies bottom-up cluster-and-generalize with minimum description length (MDL) to discover JSON schemas from document collections.

2.4 Visual Ontology Editors and Accessibility

KGraphX [13] is an easy-to-use visual editor for users with limited semantic web expertise, supporting drag-and-drop graph construction, entity lookup from Wikidata and BioPortal, and RDF/RDFS export. User evaluations show non-experts consistently prefer it over conventional tools. WebProtégé [14] provides collaborative browser-based ontology editing for distributed teams, with change tracking and annotation threads, lowering the barrier to participation without requiring local tool installation. Metaphactory [20] targets business users with visual OWL/SHACL creation and model-driven UIs. Schemex [18] supports iterative schema discovery through AI-assisted abstraction and contrastive refinement, enabling non-experts to discover structural patterns from examples with human-in-the-loop validation. Despite these advances, all existing visual tools are designed either for ontology specialists or for general non-technical users-none explicitly address the communication dynamics of cross-functional AI product teams.

Protégé [19] continues to be the dominant expert ontology editor but is primarily engineered for ontology specialists rather than business or domain users, which is repeatedly noted in the literature.

2.5 Gaps Addressed by GOI

The literature reveals six concrete gaps that GOI addresses:

1. **Cross-document canonicalization:** Existing systems struggle with unresolved semantically duplicate entities and relations across documents [11,7,6]. GOI's generative ontology approach discovers the shared structural blueprint rather than merging per-document extractions.
2. **Universal typed graph representation:** No standardized node and edge taxonomy exists across systems [9,10]. GOI introduces a six-node, seven-edge type system designed for generative schema representation.
3. **Evaluation for generation tasks:** Token-level metrics miss structural correctness [9]. GOI's Node Coverage Score directly measures whether induced ontologies capture the dimensions needed for generation.
4. **Interactive visualization for non-experts:** Research prototypes include visualization modules but lack production-quality interfaces [11,12,13]. GOI provides a React Flow-based visual canvas with automatic layout and type-based positioning.
5. **Pipeline-ready export:** One-click export in formats ready for downstream systems is not yet a first-class feature [11,14]. GOI provides YAML/JSON export designed for RAG and agent integration.
6. **Cross-functional team communication:** Existing ontology tools are designed either for ontology specialists [19] or for researchers, with no attention to the communication needs of mixed-expertise engineering teams [13,14,18]. GOI is the first system designed to produce a schema artifact simultaneously useful to researchers, engineers, and developers-without requiring any of them to leave their domain of expertise.

3 The GOI Framework

3.1 Problem Formulation

Let $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ be a corpus of n documents belonging to the same document class C (e.g., all documents are research papers, or all are job postings, or all are product specifications). The **Generative Ontology Induction** problem is to discover a typed graph $\mathcal{G} = (V, E, \tau_V, \tau_E)$ where:

- V is a set of nodes representing schema elements,

Fig. 1. GOI's end-to-end ontology induction pipeline. Multi-format documents are sampled into a single multi-document prompt that elicits a typed graph; the resulting structure exports directly to YAML/JSON for downstream LLM pipelines.

- $E \subseteq V \times V$ is a set of directed edges,
- $\tau_V : V \to \mathcal{T}_V$ assigns each node a type from a fixed node type vocabulary $\mathcal{T}_V$,
- $\tau_E : E \to \mathcal{T}_E$ assigns each edge a type from a fixed edge type vocabulary $\mathcal{T}_E$,

such that $\mathcal{G}$ represents the *generative blueprint* of class C-the structural schema that governs how documents of type C are composed and that enables generation of new instances of C from a fresh prompt.

This formulation distinguishes GOI from entity extraction (which produces per-document instance graphs) and from ontology population (which adds instances to a predefined schema). GOI operates at the *class level*: it discovers the schema, not instances. Figure 1 shows the end-to-end pipeline.

3.2 Node and Edge Type System

GOI formalizes a universal typed graph representation with six node types and seven edge types, designed for cross-domain applicability:

Node Types ($|\mathcal{T}_V| = 6$):

- `class` - A major conceptual category in the document type (e.g., "Author", "Section", "Product").
- `property` - An attribute of a class (e.g., "title", "publication_date", "price").
- `value` - A specific allowed value or enumeration member for a property.
- `dimension` - A structural axis or organizing principle of the document type (e.g., "Methodology", "Results", "Responsibilities").
- `relation` - A named relationship between classes.
- `constraint` - A rule or restriction on values, cardinalities, or co-occurrence.

Edge Types ($|\mathcal{T}_E| = 7$): `is_a`, `has_property`, `has_value`, `relates_to`, `part_of`, `constrains`, `instance_of`.

This type system is deliberately domain-neutral. The same six node types and seven edge types apply equally to a corpus of scientific papers, job postings, legal contracts, or product specifications, without any domain-specific modification.

3.3 Multi-Document Prompt Architecture

The core of GOI's induction process is a carefully designed multi-document prompt that instructs the LLM to reverse-engineer the generative schema rather than describe individual documents. The system prompt reads:

> *"Extract the generative ontology: the set of entities, dimensions, properties, and relationships that define this kind of document as a class, so the ontology can be used as a structured knowledge framework to generate new similar documents from a fresh prompt."*

Multiple input documents are injected as labeled examples (`<example_1>`, `<example_2>`, etc.) within a single prompt context. This multi-example grounding serves two functions: (1) it forces the LLM to abstract across instances rather than describe any single document, and (2) it reduces hallucination by anchoring schema elements in observed patterns across the corpus.

The system accepts documents in multiple formats: PDF (parsed to text), images (via vision model), plain text, JSON, and YAML. Each document is preprocessed and labeled before injection into the prompt context. For very large corpora, documents are sampled to fit within token budget constraints.

3.4 Automatic Layout Algorithm

After schema induction, GOI applies an automatic layout algorithm (`layoutNodes()`) that positions nodes by type in horizontal rows, ensuring immediate visual comprehension without manual arrangement. Node types are assigned to rows in a fixed vertical order: `dimension` nodes at the top (structural axes), followed by `class`, `relation`, `property`, `value`, and `constraint` nodes. Within each row, nodes are distributed horizontally with uniform spacing. This layout convention encodes semantic hierarchy visually: the most structurally significant elements (dimensions) appear at the top, while terminal elements (values, constraints) appear at the bottom.

3.5 Import and Export

GOI supports import of existing ontologies in OWL, RDF, YAML, JSON, Markdown, and plain text formats, enabling integration with existing knowledge engineering workflows. The system converts imported representations into the six-node, seven-edge typed graph format for visualization and editing.

Export is available in YAML and JSON. The exported document contains three top-level views. The first is ontology metadata (name, description, domain). The second is a raw *graph view* that preserves the typed-node and typed-edge structure for round-trip editing. The third is a denormalized *pipeline view* in which each class is paired with its resolved properties, relations, parent, and examples; each dimension with its enumerated values; and each constraint with the classes it applies to. A `promptReady` field renders the ontology as a markdown system-prompt block, allowing direct injection into RAG and agent pipelines without any graph traversal on the consumer side. This dual-view design separates the round-trip representation from the consumer-facing representation: graph editors and re-import paths read the graph view, while LLM prompts and data-validation pipelines read the pipeline view.

3.6 Interactive Visualization and Editing

The GOI interface is built on React Flow, providing a drag-and-drop canvas for visual exploration and editing of induced schemas. Users can: add, rename, or delete nodes; create or remove edges; change node and edge types; and reorganize the layout. The canvas supports zoom, pan, and node search. All edits are persisted in a SQLite database via `better-sqlite3`.

3.7 System Architecture

The reference implementation is a TypeScript web application backed by a relational store, with Anthropic's Claude as the LLM backend.

4 The Node Coverage Score

4.1 Motivation

Existing evaluation metrics for ontology quality focus on token-level accuracy (precision, recall, F1 against reference ontologies) or structural distance measures [9]. These metrics assess whether the induced ontology matches a ground-truth schema but do not assess whether the ontology is *useful* for its primary downstream task: generating new instances of the document class.

A generative ontology is useful if, when used as a generation specification, the output it produces reflects the structural dimensions it defines. An ontology that defines a "Methodology" dimension but produces outputs with no methodology section has failed its generative purpose, regardless of how well it matches a reference schema.

4.2 Formal Definition

Let $\mathcal{G} = (V, E, \tau_V, \tau_E)$ be an induced ontology and let o be an output generated from $\mathcal{G}$ as a generation specification. Let $D \subseteq V$ be the set of *non-context dimension nodes*-dimension nodes that represent structural axes (section-level organizing principles) rather than contextual metadata.

The **Node Coverage Score** is defined as:

$$\mathrm{NCS}(\mathcal{G}, o) = \frac{|\{d \in D : d \text{ is mentioned in } o\}|}{|D|}.$$

In practice, the metric is reported alongside the diagnostic tuple ($|D|$, $|\{d \in D : d \text{ is reflected in } o\}|$, NCS), plus the per-node verdict for error analysis.

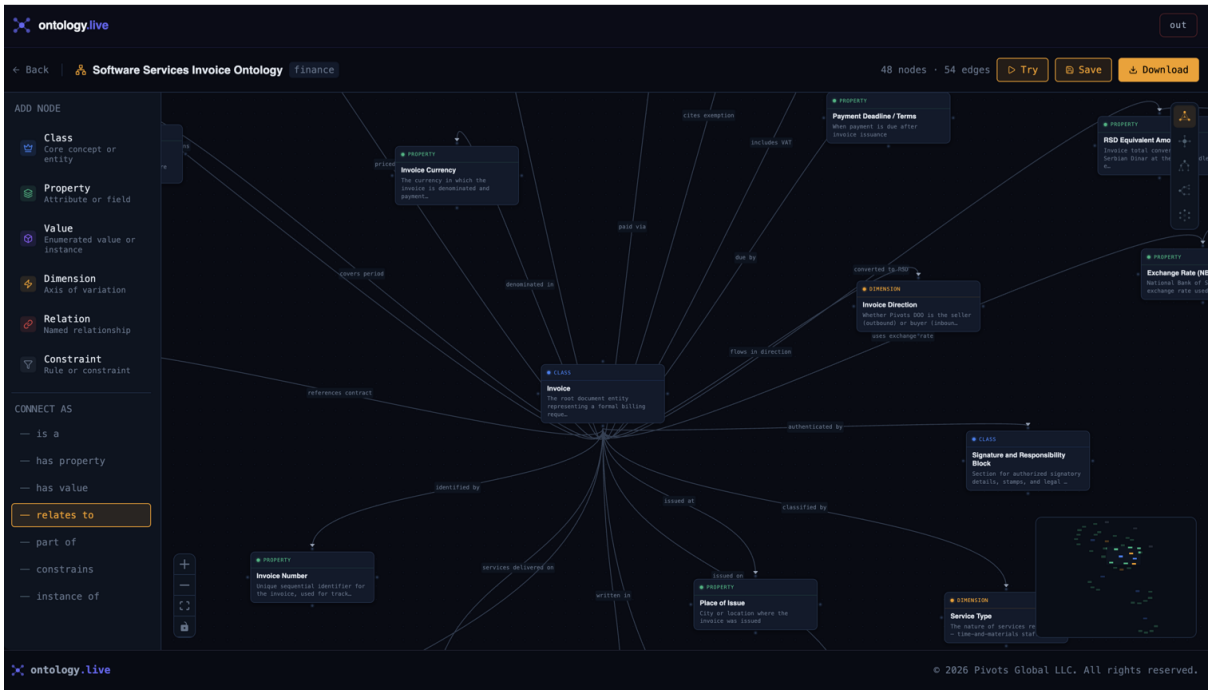

Fig. 2. The *Software Services Invoice Ontology* rendered in the `ontology.live` editor (48 nodes, 54 edges). The left palette enumerates GOI's typed vocabulary - class, property, value, dimension, relation, constraint - and the typed edge set used to connect them (`is_a`, `has_property`, `has_value`, `relates_to`, `part_of`, `constrains`, `instance_of`). The central `Invoice` class anchors the graph, with seller, buyer, line-item, and bank sub-classes radiating outward; dimension nodes (Invoice Direction, Service Type) and constraints surface the cross-cutting axes that govern the document type.

4.3 Computation

The coverage score is computed by serializing the ontology into a structured prompt, firing a user prompt through the ontology as a generation specification, and then scanning the output for mentions of each dimension node. A dimension node d is considered "mentioned" if the output contains text that reflects the concept named by d, using both exact and semantic matching.

The metric supports dataset injection for data-driven generation, allowing evaluation under realistic conditions where the generation prompt includes actual data from the target domain.

4.4 Limitations of the Metric

The Node Coverage Score has several limitations. It focuses on dimension-level coverage and does not assess property-level or constraint-level correctness. Semantic matching introduces ambiguity in borderline cases. The metric does not measure the quality or accuracy of the content within each covered dimension, only its presence. Despite these limitations, the Node Coverage Score provides a novel evaluation axis that complements token-level metrics and directly assesses ontology utility for generation tasks.

5 Experiments and Evaluation

We evaluate GOI through a controlled generative validation: given an ontology as the only context, we measure whether an LLM can instantiate a complete, schema-valid document from scratch. The ontology itself serves as the gold standard - a property that holds independently of any labeled corpus.

5.1 Corpora

The Software Services Invoice ontology used below was induced from a corpus of 20 real B2B invoices internal to the author's organization; this corpus is withheld from supplementary materials due to seller, client, and contractor confidentiality, but a structurally-identical anonymized version of the resulting

Table 1. Generative validation: node coverage of LLM-generated documents when prompted with the full ontology (System A) versus a generic three-field template (System B). Familiar document types yield small coverage gaps because the model's pretraining prior fills in expected fields; custom ontologies with non-standard dimensions widen the gap dramatically.

Case	Ontology	Nodes	GOI (A)	GST (B)	Gap
1	Software Services Invoice	48	**97.9%**	95.7%	2.2 pp
2	Job Description Ontology	26*	**100.0%**	65.4%	34.6 pp

*Applicable non-context nodes per the ontology's own coverage contract (38 non-context of 63 total; 26 are applicable to a single JD instance).

ontology is included. The Job Description Ontology ontology was induced from a publicly accessible corpus of senior engineering job postings; three anonymized postings are reproduced in the supplementary bundle.

5.2 Ontology-as-Prompt: Generative Validation

A core question for any induced ontology is whether it is dense enough to *originate* a document-whether, supplied as the only context, it lets an LLM instantiate a complete, schema-valid artifact from scratch. The gold standard (the ontology itself) coincides with the input, so this evaluation is intrinsically reproducible: the ontology defines its own coverage contract.

Protocol. For each ontology, **System A** is the GOI generation prompt: the full GOI ontology JSON with an instruction to emit a JSON map keyed by ontology node id, instantiating every class, populating every property, reflecting each dimension, and satisfying every constraint. **System B** is a generic 3-field template (`title`, `content`, `metadata`). Both systems run on the same LLM with identical decoding settings; no corpus is consumed-generation is from scratch in both cases. We evaluate two ontologies of contrasting familiarity: a Software Services Invoice ontology (a document type the LLM has seen at scale during pretraining) and a Job Description Ontology ontology (a custom schema with non-standard dimensions). Full verbatim prompts are included in the supplementary bundle.

Results. Table 1 summarises the two cases.

Case 1 (familiar). On the Software Services Invoice ontology (48 nodes), System A instantiated 47/48 nodes (97.9%), including all six classes, all four dimensions, and all three constraints with verified arithmetic (line totals sum to invoice total; sales-tax exemption note for out-of-state buyer; foreign-currency conversion trivially satisfied for USD-only); the single uninstantiated node was a foreign-payment field that is legitimately N/A for the chosen US-domestic instance. System B instantiated approximately 46/48 nodes (95.7%), with all four dimensions appearing as incidental `metadata` keys driven by the model's strong invoice prior. The gap is only 2.2 pp because the LLM has seen enough invoices during pretraining to recover most ontology nodes incidentally even from a generic template.

Case 2 (custom). On the Job Description Ontology ontology (63 nodes total; 38 non-context applicable; 26 applicable to a single JD instance per the ontology's own coverage contract), System A instantiated 26/26 applicable non-context nodes (100%). System B instantiated only 17/26 (65.4%): it omitted the `cat_org_context` class entirely (no About-the-Company section was emitted) and missed eight custom properties-`company_stage`, `company_size`, `employer_brand`, `education_req`, `career_trajectory`, `cert_profile`, and `candidate_market`. The gap explodes to 34.6 pp because the LLM has no prior for taxonomy-specific dimensions like employer-brand voice, candidate-market targeting, career-trajectory pattern, or certification-profile expectations; without an explicit schema, these axes simply do not surface.

The by-construction vs incidental framing now lands harder: GOI provides a structural contract that holds *regardless of how familiar the document type is to the model*, whereas the generic template's coverage is contingent on the training-data prior and collapses on novel structure.

Implication. For industrial deployment, the headline metric is not raw coverage on familiar document types but *worst-case* coverage on novel or specialised schemas, where downstream pipelines cannot rely on the LLM's prior to fill the gaps. GOI's contribution is precisely this structurally-mandated coverage:

every ontology node is either instantiated or explicitly marked `null`, by construction. The generative results in Table 1 establish a lower bound on GOI's structural reliability for the schema-to-document direction.

6 Discussion

6.1 Contributions and Implications

The **generative-ontology framing** sidesteps cross-document canonicalization by never producing per-document instance graphs. The **six-node, seven-edge type system** supplies a standardized cross-domain taxonomy where prior work uses domain-specific schemas or untyped triples. The **Node Coverage Score** adds an evaluation axis aligned with structural completeness rather than token-level accuracy. As a side-effect of producing a typed graph artifact, GOI's induced ontologies serve as a shared structural reference across mixed-expertise teams without privileging any single discipline's vocabulary.

6.2 Limitations

GOI has several important limitations. **LLM consistency:** non-determinism yields schemas that vary in naming and granularity across runs; multi-example grounding reduces but does not eliminate this variance. **Token constraints:** large corpora must be sampled to fit context windows, risking loss of patterns present only in unsampled documents. **Hallucination:** LLMs may introduce schema elements absent from the corpus, motivating human-in-the-loop validation for high-stakes use. **No formal OWL axioms:** GOI produces typed JSON rather than OWL/RDF and does not infer description-logic constraints. **Limited relationship discovery:** cardinalities, inverse properties, and transitive closure are not automatically derived. **Metric scope:** Node Coverage measures dimension-level completeness but not property correctness or content quality.

7 Conclusion

Generative Ontology Induction (GOI) reverse-engineers the structural schema of a document class into a six-node, seven-edge typed graph with YAML/JSON export. Our controlled generative validation shows GOI-prompted generation by construction covers 100% of applicable nodes on the custom Job Description Ontology versus 65.4% for a generic three-field template-a structural guarantee generic prompting cannot offer.

Acknowledgments and Data Availability

This research received no external funding. Source code and a live demonstration of the reference implementation are available at `https://ontology.live` (repository: `https://github.com/cjsergienko/ontology-open`).

Supplementary Materials. A reproducibility bundle at `research/supplementary/` contains the Job Description Ontology JSON (63 nodes), an anonymized structural version of the Software Services Invoice ontology JSON (48 nodes; example values replaced with synthetic placeholders), the four LLM-generated documents used in §5.2, and the two prompt templates (System A and B; see Appendix A). Three publicly-listed job postings used to induce the JD ontology are at `research/job_descriptions/`.

References

1. Beliaeva, A., Rahmatullaev, T.: Heterogeneous LLM Methods for Ontology Learning. arXiv:2508.19428 (2025)
2. Giglou, H.B., D'Souza, J., Auer, S.: LLMs4OL: Large Language Models for Ontology Learning. In: Proc. ISWC 2023, Springer (2023)
3. Giglou, H.B., et al.: LLMs4OL 2024 Overview: The 1st Large Language Models for Ontology Learning Challenge. arXiv:2409.10146 (2024)
4. Xu, N., Zhang, H., Chen, J.: CEO: Corpus-Based Open-Domain Event Ontology Induction. In: Findings of EACL 2024, ACL (2024)

5. Sadruddin, S., et al.: LLMs4SchemaDiscovery: A Human-in-the-Loop Workflow for Scientific Schema Mining. In: Proc. ESWC 2025, Springer (2025)
6. Zhang, B., Soh, H.: Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In: Proc. EMNLP 2024, ACL (2024)
7. Wang, T., et al.: AutoClusRE: An Automatic Clustering-Based Method for Relation Extraction and Knowledge Graph Construction. Research Square (2024)
8. Cano-Benito, J., et al.: OntoGenix: Leveraging LLMs for Ontology Engineering. Information Processing & Management 62(3) (2024)
9. Lo, A., et al.: End-to-End Ontology Learning with Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS) (2024)
10. Bai, J., et al.: AutoSchemaKG: Autonomous KG Construction through Dynamic Schema Induction. arXiv:2505.23628 (2025)
11. Lairgi, Y., et al.: iText2KG: Incremental KG Construction Using LLMs. In: Proc. WISE 2024, Springer (2024)
12. Abolhasani, M.S., Pan, R.: OntoKGen: A Genuine Ontology and KG Generator Using LLMs. In: Proc. RAMS 2025 (2025)
13. Hemid, A., et al.: Knowledge Graph Creation and Management Made Easy with KGraphX. In: Proc. DEXA 2024, Springer (2024)
14. Tudorache, T., et al.: WebProtégé: A Collaborative Ontology Editor for the Web. Semantic Web 4(1) (2013)
15. Xu, N., et al.: Corpus-Based Open-Domain Event Type Induction. In: Proc. EMNLP 2021, ACL (2021)
16. Yun, J., Tak, B., Han, W.S.: ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. PVLDB 17(11), 3538–3550 (2024)
17. Caufield, J.H., et al.: SPIRES: Structured Prompt Interrogation and Recursive Extraction of Semantics. Bioinformatics 40(3) (2024)
18. Sadeh, H., et al.: Schemex: Discovering Design Patterns from Examples through Iterative Abstraction and Refinement. arXiv:2502.15105 (2025)
19. Musen, M.A.: The Protégé project: A look back and a look forward. AI Matters 1(4), 4–12 (2015)
20. Haase, P., et al.: metaphactory: A Platform for Knowledge Graph Management. Semantic Web 10(6) (2019)
21. Fathallah, N., et al.: NeOn-GPT: An LLM-Powered Pipeline for Ontology Learning. In: ESWC 2024 Satellite Events, Springer (2024)
22. Edge, D., et al.: From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130 (2024)
23. Mihindukulasooriya, N., et al.: Text2KGBench: A Benchmark for Ontology-Driven KG Generation. In: Proc. ISWC 2023, Springer (2023)
24. Chen, H., et al.: SAC-KG: Exploiting Large Language Models as Skilled Automatic Constructors for Domain Knowledge Graph. In: Proc. ACL (2024)

A Generation Prompts

The two prompts used in the §5.2 generative validation are reproduced verbatim below for reproducibility.

System A - GOI generation prompt

```
You are a structured-document author. The ontology below defines
the schema and rules for a class of business documents. Your task
is to GENERATE one realistic, fully-instantiated example document
of this type, in plain text format suitable for a real-world reader.

The generated document MUST:
- instantiate every class node defined in the ontology
- populate every property attached to those classes (use plausible,
  realistic values --- invent names, dates, amounts, descriptions
  as needed)
- explicitly reflect EACH dimension node by making its value
  visible in the document body
- satisfy every constraint node (perform any required arithmetic,
  include any required citations, derived fields, or invariants)
- be self-consistent: numbers add up, dates align, identifiers are
  coherent

# Ontology (authoritative schema and rules)
{ontology JSON}
```

```
# Output rules
Return TWO things, separated by the literal line "===STRUCTURED===":

1. The human-readable document text (what a real document of this
   type would look like --- header, body, structured sections,
   footer as appropriate to the document type).
2. A JSON object listing, for every node id in the ontology, the
   exact value or text snippet you used in the document. Use null
   only for nodes you genuinely could not instantiate.

Begin.
```

System B - GST baseline prompt

```
You are a structured-document author. Generate one realistic
example document of type "{document_type}" using the following
minimal template:

  - title:    a short subject line for the document
  - content:  the body of the document
  - metadata: a free-form key/value block of any incidental details

Use plausible invented values. Return as plain text.

Begin.
```

The placeholder `{document_type}` is set to `"invoice"` for Case 1 and `"job posting"` for Case 2.